

DEMANDSHOCK

Crisis-Aware Demand Forecasting & Inventory Intelligence Platform

Technical Project Report

Design, implementation, measured results and independent verification

Attribute	Value
Platform version	1.0.0
Run mode	Full dataset (all 30,490 item-store series)
Primary model	Global LightGBM (Tweedie), one model per horizon bucket
Selected feature set	B (+ Calendar & Price)
Data coverage	2011-01-29 to 2016-06-19
Model trained	2026-08-05T00:19:03+00:00
Configuration hash	f87039f3db9b
Automated tests	135 passing, 0 skipped
Verification status	Independently audited (read-only)

Contents

Contents	2
1. Executive Summary.....	5
1.1 Headline measured results	5
1.2 The most important finding.....	5
2. Business Problem and Proposed Solution.....	7
2.1 The problem	7
2.2 The proposition	7
2.3 Questions the platform answers	7
2.4 Concentration of value.....	7
3. Data Sources and Integrity.....	8
3.1 Sources	8
3.2 Volume actually processed	8
3.3 Data quality validation.....	8
3.4 Deliberate exclusions	8
3.5 Handling of pre-assortment days	9
4. System Architecture.....	10
4.1 Pipeline stages.....	10
4.2 Data flow	10
4.3 Technology choices	10
4.4 Verified operating environment.....	11
5. Forecasting Methodology	12
5.1 The governing invariant	12
5.2 Why a direct model rather than a recursive one	12
5.3 Feature specification.....	12
5.4 Objective function	12
5.5 Validation design.....	13
5.6 Metric definitions.....	13
5.7 Horizon buckets: giving each step the freshest data it may use	14
6. Measured Results.....	15
6.1 Model comparison	15
6.2 Accuracy by forecast horizon.....	15
6.3 Prediction intervals	15
6.4 Feature attribution.....	16
7. Ablation Study: Do External Signals Help?	17

7.1 Arms.....	17
7.2 Results	17
7.3 Per-fold detail.....	17
7.4 Verdict	17
8. Demand Shock Intelligence.....	19
8.1 Why the residuals come from a crisis-free model	19
8.2 Score construction.....	19
8.3 Guards against false positives	19
8.4 Severity bands	19
8.5 Measured detection results.....	20
8.6 Real-world context, stated as coincidence.....	20
8.7 Auditable explanations	20
9. Inventory and Business Impact.....	21
9.1 Measured from real data	21
9.2 Requires user-supplied operating assumptions.....	21
9.3 Separating technical results from business assumptions	22
10. Reading a Forecast Correctly	23
10.1 Why a point forecast cannot track a spiky series.....	23
10.2 The same forecast, judged at three grains	23
10.3 Uncertainty is the answer, not the point forecast	23
10.4 Accuracy is not uniform across the assortment.....	24
11. Product Identification	25
11.1 Enforcement	25
12. Application and Service Layer.....	26
12.1 The five modules	26
12.2 Interface principles	26
12.3 Service layer	26
13. Engineering Quality and Verification.....	27
13.1 Test suite	27
13.2 The central leakage proof	27
13.3 Independent verification	27
14. Limitations and Responsible Use.....	29
14.1 Data limitations.....	29
14.2 Methodological limitations.....	29
14.3 Responsible use.....	29
15. Reproduction.....	30

15.1 Environment 30

15.2 Build and serve 30

15.3 Measured run characteristics 30

16. Conclusion 30

1. Executive Summary

DemandShock is a decision-support platform for retail demand planning. It forecasts daily demand for every product in every store, detects where actual demand has broken away from what the model expected, attaches the real-world conditions that coincided with those breaks and translates the forecasts into transparent inventory planning arithmetic.

The distinguishing idea is that the forecast residual is treated as a signal in its own right, not merely as an error term. Conventional forecasting tools report an average accuracy figure. They do not tell a planner which specific series broke, how badly, what else was happening at the time or what it is worth in dollars. DemandShock answers those questions.

1.1 Headline measured results

Result	Value	Basis
Series modelled	30,490	M5 item x store
Item-days processed	46,881,677	after release filtering
Forecast accuracy (WAPE)	0.7198	28-day held-out window
Scaled error (RMSSE)	0.7550	below 1.0 beats naive
Improvement vs seasonal naive	19.1%	same information set
Prediction interval coverage	79.4%	target 80%, n=853,624
Near-term accuracy (steps 1-7)	0.7226	was 0.7519
Weekly-grain WAPE	0.3683	same forecast, weekly totals
Total-demand WAPE	0.0684	all series summed per day
Demand shock episodes detected	52,922	out-of-sample window
Estimated revenue exposure	\$3,046,781	real prices, 28 days
Data quality checks	63 / 63 passed	0 warnings, 0 failures

1.2 The most important finding

The project set out to test whether external crisis and economic context improves demand forecasting. Measured honestly across identical rolling-origin folds, the answer is no.

Adding FEMA disaster context changed WAPE by **0.0002 (0.03% relative)**, which is smaller than the fold-to-fold variation of **0.0088**. Adding FRED economic context made accuracy slightly worse. The selection rule therefore chose the simpler feature set and both external sources were retained for explaining shocks rather than for point-forecast accuracy.

This is reported prominently inside the application itself, auto-generated from the numbers rather than written by hand. A platform that quietly buried this result would be less trustworthy, not more.

What this platform is and is not

It is decision support: forecasts, deviation detection, context and planning arithmetic for a human planner. It is not an autonomous replenishment system, it does not claim causation between disasters and demand and it never presents an assumption as an observation.

2. Business Problem and Proposed Solution

2.1 The problem

Retail planners forecast adequately in ordinary conditions. The expensive failures happen when demand stops behaving normally: a product's demand regime shifts, a category surges, a store's pattern breaks. These events are often not noticed for weeks, because a portfolio-level accuracy metric absorbs them.

Two costs follow. Under-forecasting risks lost sales and empty shelves. Over-forecasting ties up working capital in stock that does not move. Both are invisible in an aggregate error figure and both are concentrated in a small number of series.

2.2 The proposition

Retail intelligence for when historical demand patterns stop behaving normally.

Traditional forecasting asks what demand is likely to be. DemandShock also asks where actual behaviour is deviating from expectation, what real external conditions coincided with that change, how important the deviation is and what action a planner should consider.

2.3 Questions the platform answers

Question	Where it is answered
What is likely to happen to demand?	Module 2 - forecasts with P10-P90 intervals
Where is demand behaving abnormally?	Module 3 - demand shock scoring
Did deviations coincide with real disruptions?	Module 3 - FEMA and FRED context
Which products and stores need attention?	Modules 1 and 3
What inventory does the forecast imply?	Module 4 - planning arithmetic
How reliable is the model?	Module 5 - backtests, ablation, intervals
What drives the forecasts?	Module 5 - exact TreeSHAP attribution
What financial exposure does error create?	Modules 1 and 4

2.4 Concentration of value

The exposure analysis shows the practical payoff of prioritisation. Across 30,490 series in the held-out window, forecast error carried an estimated \$3,046,781 of revenue exposure, of which \$1,656,859 came from under-forecasting and \$1,389,922 from over-forecasting. Because that exposure is heavily concentrated, a planner who reviews a small ranked list addresses a disproportionate share of the risk.

3. Data Sources and Integrity

The platform uses only real, publicly documented datasets. No synthetic records, simulated crises, fabricated inventory positions or placeholder metrics exist anywhere in the pipeline, the application or this report.

3.1 Sources

Source	Contribution	Grain
M5 Forecasting - Accuracy	Daily unit sales, weekly sell prices, retail calendar with events and SNAP flags	Item x store x day
FEMA OpenFEMA v2	Disaster declarations: type, incident type, dates, designated counties	Disaster x county
FRED / BLS	Monthly state unemployment for California, Texas and Wisconsin	State x month

3.2 Volume actually processed

Quantity	Value
Item-store series	30,490
Distinct items / stores	3,049 / 10
Categories / departments / states	3 / 7 / 3
Item-days after release filtering	46,881,677
Weekly price records	6,841,121
Calendar days covered	1,969 (2011-01-29 to 2016-06-19)
Zero-sale share of item-days	59.6%
FEMA disasters in window (CA/TX/WI)	116
FRED monthly observations	1,818

Demand is highly intermittent: **59.6%** of item-days record zero sales. This single fact drives most of the modelling decisions that follow, from the choice of objective function to the way metrics are interpreted and the guards built into shock detection.

3.3 Data quality validation

The ingestion stage runs 63 automated checks and refuses to proceed if any fails. On the reported run: 63 passed, 0 warnings, 0 failures. Coverage includes file and column schemas, calendar continuity, hierarchy consistency, duplicate keys, price validity, FEMA deduplication counts, FRED coverage and join integrity.

3.4 Deliberate exclusions

- NOAA GHCN-Daily weather extracts are present on disk but never read. Weather is out of scope in this version and is documented as a possible extension, no feature, page or endpoint depends on it.

- The Kaggle submission scaffold and spreadsheet duplicates of the FRED series are likewise excluded and enumerated in configuration.

3.5 Handling of pre-assortment days

A product that had not yet been introduced in a store records zeros that are not demand signal. Rows before a series' first priced week are therefore dropped rather than treated as zero demand. The rule is applied consistently and the number of removed rows is counted and reported in the data quality artifact, never applied silently.

4. System Architecture

The system is a batch pipeline that produces artifacts, plus two thin read-only presentation layers. The application and the API never train models, never read raw CSV files and never load a feature matrix. This decoupling is what keeps the user interface responsive regardless of the scale of the training run.

4.1 Pipeline stages

Stage	Script	Produces
1. Ingest and validate	prepare_data.py	Parquet tables, data quality report
2. Feature engineering	prepare_data.py	Model-ready feature matrix per store
3. Train and backtest	train.py	Forecasts, metrics, ablation, intervals
4. Shock and inventory	evaluate.py	Shock scores, episodes, exposure table
5. Publish results	export_results.py	RESULTS.md generated from artifacts

4.2 Data flow

M5 sales, prices and calendar, together with FEMA declarations and FRED unemployment, pass through validation and normalisation into Parquet tables. Feature engineering produces an origin-frozen feature matrix. Three benchmark models and one global LightGBM model are trained under rolling-origin backtesting, the ablation compares four cumulative feature sets. Residuals feed both the empirical prediction intervals and the shock detector. The deploy model produces a genuine forward forecast beyond the end of the data. All outputs land in an artifact directory that the Streamlit application and FastAPI service read.

4.3 Technology choices

Component	Choice	Reason
Data processing	pandas, numpy, pyarrow	Sufficient at this scale, no extra dependencies
Model	LightGBM (Tweedie)	Intermittent counts, native categoricals, exact TreeSHAP
Uncertainty	Empirical residual quantiles	Calibrated by construction, near-zero cost
Application	Streamlit + Plotly	Rapid, dark analytical interface
Service layer	FastAPI + pydantic	Thin typed read layer over the same artifacts
Testing	pytest	Leakage proofs are release-gating

Deliberately not built

No orchestration cluster, message queue, feature store, experiment tracker or microservice layer. No recursive forecaster, no additional model families, no synthetic scenario simulator and no causal inference. Each omission was a decision to protect correctness and clarity rather than an oversight.

4.4 Verified operating environment

Library	Version
python	3.14.4
pandas	3.0.2
numpy	2.4.4
lightgbm	4.6.0
pyarrow	23.0.1
scipy	1.17.1
scikit-learn	1.8.0

5. Forecasting Methodology

5.1 The governing invariant

Every feature for target date t is computable from information available at the forecast origin, defined as t minus 28 days or is genuinely known in advance.

Three consequences follow. Demand lags begin at 28 days. Rolling statistics are computed on a series already shifted by 28 days, so no window can reach into the forecast horizon. FEMA and FRED tables are joined at the origin date rather than the target date, so they represent the state of the world as it was known when the forecast was made.

5.2 Why a direct model rather than a recursive one

A recursive one-step model would feed its own predictions back into lag features. With the majority of item-days at zero, those predictions are fractional values entering features whose training distribution is integer and zero-heavy. That distribution shift compounds across 28 steps and would contaminate precisely the residuals the shock detector depends on. A direct model also means training and inference share one feature function, so there is no second implementation to drift out of sync.

The honest cost is stated inside the application: a seven-day-ahead forecast sees demand only as of 28 days earlier. This is also why seasonal-naïve at lag 28 is the primary benchmark - it forecasts from exactly the same age of information.

5.3 Feature specification

The full specification contains 47 features across six families. The selected feature set B uses 36 of them.

Family	Count	Representative features
Demand history	14	Lags 28-56, rolling mean and standard deviation over 7/28/56 days, zero rate, days since last sale
Calendar	12	Weekday, day of month, week of year, SNAP for the store's own state, event names and proximity
Price	5	Sell price, weekly change, price relative to a 52-week mean, 4-week momentum
FEMA context	7	Active declaration count, major-declaration flag, incident-type flags, designated-county extent
Economic context	4	Unemployment level and 1-month, 3-month, year-over-year change
Identity	5	Item, department, category, store, state as native categoricals

5.4 Objective function

The Tweedie objective was selected because it models a point mass at zero together with a continuous positive part, which matches intermittent unit sales. The choice was validated empirically against Poisson and squared-error alternatives on a single development fold and then frozen. No further hyperparameter search was performed and the application discloses this.

5.5 Validation design

Validation is chronological throughout. No random splitting exists anywhere in the codebase. Three rolling-origin folds are used for the ablation and for pooling residuals, the holdout window is opened once, after the feature set has been selected, a forward window beyond the end of the data receives a genuine forecast with no actuals to compare against.

Split	Days	Dates	Role
Fold F1	1830-1857	2016-02-01 to 2016-02-28	Ablation, residual pool
Fold F2	1858-1885	2016-02-29 to 2016-03-27	Ablation, residual pool
Fold F3	1886-1913	2016-03-28 to 2016-04-24	Ablation, residual pool
Holdout	1914-1941	2016-04-25 to 2016-05-22	Headline results, opened once
Forward	1942-1969	2016-05-23 to 2016-06-19	Forward forecast, no actuals

5.6 Metric definitions

Metric	Definition and handling
WAPE	Total absolute error divided by total actual units. The most reliable headline metric on intermittent demand.
RMSSE	Root mean squared error scaled by the in-sample one-step naive error per series. Below 1.0 beats that benchmark. Series with flat training history are excluded and the exclusions are counted.
MAE / RMSE	Mean absolute and root mean squared error in units, pooled across item-days.
sMAPE	Reported for completeness but unstable here: a zero forecast scores perfectly on a zero-sale day while any positive forecast is penalised the full 200%.
Bias	Total forecast minus total actual as a share of actual. Positive indicates over-forecasting.

A metric deliberately not claimed

The official M5 WRMSSE aggregate is not implemented, so the name is never used. An automated test scans the entire codebase to ensure no component claims it.

5.7 Horizon buckets: giving each step the freshest data it may use

A 28-day path is issued once at origin O, so a model serving steps 1..S may read data only up to O - a shift of S relative to the target date. The platform originally froze every step at 28 days so a single model could serve all horizons. That had a perverse consequence: the 1-day-ahead forecast read demand from 27 days earlier, while the 28-day-ahead forecast read the freshest day available. Accuracy was consequently flat across the horizon, the opposite of what a forecaster should do.

Bucket	Steps	Origin shift	Newest input at its last step
H1	1-7	7 days	the forecast origin
H2	8-14	14 days	the forecast origin
H3	15-28	28 days	the forecast origin

The safety rule is $\text{shift} \geq \text{max_step}$: at a bucket's last step the newest input is exactly the origin. Three automated tests enforce it - one checks every bucket satisfies the rule and that the buckets tile steps 1 to 28 exactly once and a parameterised test re-proves the perturbation invariant at each shift, each with a control showing the test cannot pass vacuously.

Measured effect

Forecast steps	Single 28-day freeze	Horizon buckets	Improvement
1-7 days ahead	0.7519	0.7226	3.9%
8-14 days ahead	0.7385	0.7233	2.1%
15-28 days ahead	0.7274	0.7274	unchanged
All 28 days (WAPE)	0.7305	0.7198	1.5%
All 28 days (RMSSE)	0.7606	0.7550	0.7%

Two controls confirm the result is real rather than an artefact. Steps 15-28 are unchanged, exactly as predicted, because their shift did not move - 28 was already the correct value for that bucket. And the naive benchmarks are identical to four decimal places, as they must be since they use no features at all. The accuracy curve now slopes upward with horizon: near-term forecasts are the sharpest, which is the behaviour a planner expects.

What the experiment showed before the change was adopted

An isolated experiment on one fold compared shifts directly. Fresher rolling windows produced the entire gain, fresher individual day-lags added nothing measurable. On intermittent demand a single day's lag is noise where a rolling mean is signal. The change is therefore a per-bucket shift, not a feature redesign - a smaller and far safer edit.

The ablation was re-run under bucketing rather than assuming the earlier verdict still held. Every arm improved by a similar margin and the conclusion was unchanged, as reported in the ablation chapter.

6. Measured Results

All figures below come from the held-out window 2016-04-25 to 2016-05-22, which was excluded from training entirely and opened once after the feature set had been chosen.

6.1 Model comparison

Model	WAPE	RMSSE	MAE	RMSE	Bias	sMAPE
LightGBM (set B)	0.7198	0.7550	1.0385	2.1493	-0.0655	139.8
Seasonal naive (lag 7)	0.8622	0.9976	1.2440	2.6769	-0.0736	83.1
Seasonal naive (lag 28)	0.8900	1.0338	1.2840	2.8292	-0.0391	84.0
Naive (last observed day)	0.9516	1.0017	1.3730	2.8936	0.1319	85.7

LightGBM reduces WAPE by **19.1%** against seasonal-naive at lag 28, the benchmark that forecasts from the same 28-day-old information. Its RMSSE of **0.7550** is below 1.0, meaning it also beats a one-day naive forecast measured against each series' own training history.

The sMAPE column illustrates why metric choice matters on intermittent demand. The naive models score better on sMAPE precisely because they predict exact zeros, which are scored as perfect on the majority of days. WAPE and RMSSE are the trustworthy comparisons and the application labels sMAPE accordingly wherever it appears.

6.2 Accuracy by forecast horizon

Horizon	WAPE	RMSSE	Bias
7 days	0.7142	0.6707	-0.0259
14 days	0.7176	0.7162	-0.0308
28 days	0.7198	0.7550	-0.0655

6.3 Prediction intervals

Uncertainty is expressed as empirical P10-P90 intervals built from out-of-sample residuals, grouped by store, category and forecast level, with fallback tiers for sparse groups. Coverage is measured on the untouched holdout and reported whatever the outcome.

Interval property	Measured
Design target	80%
Empirical coverage	79.4%
Actuals below P10	9.7%
Actuals above P90	10.8%
Evaluation points	853,624

Coverage of 79.4% against an 80% design target, with the two tails balanced at 9.7% and 10.8%, indicates well-calibrated intervals at scale.

6.4 Feature attribution

Attribution uses exact TreeSHAP values computed natively by LightGBM. The optional shap package is not required and its absence changes nothing about what is shown.

Feature family	Share of model gain
Demand history	87.79%
Product / store identity	9.17%
Calendar and events	1.87%
Price	1.17%

The top individual drivers by gain are roll_mean_56, roll_mean_28, item_id, roll_std_56, roll_mean_7. Recent demand history dominates, which is the expected and correct outcome for short-horizon retail forecasting. Attribution measures reliance, not direction: the values are unsigned and the application states this rather than asserting a direction the artifacts cannot support.

7. Ablation Study: Do External Signals Help?

This is the study the project was designed around. Four cumulative feature sets were evaluated across identical rolling-origin folds with the same seed, parameters, training rows and eligible series. The only difference between arms is which features are present.

7.1 Arms

Arm	Feature set	Features
A	Demand history and identity only	19
B	Plus calendar and price	36
C	Plus FEMA disaster context	43
D	Plus FRED economic context	47

7.2 Results

Mean across folds F1 to F3 at the 28-day horizon.

Arm	WAPE	SD across folds	RMSSE	Bias
A	0.7591	0.0076	0.7390	-0.0492
B	0.7472	0.0088	0.7303	-0.0300
C	0.7470	0.0096	0.7301	-0.0299
D	0.7473	0.0093	0.7298	-0.0225

7.3 Per-fold detail

Fold-level results show whether a difference between arms is consistent or simply noise.

Arm	F1	F2	F3
A	0.7657	0.7607	0.7508
B	0.7516	0.7529	0.7371
C	0.7519	0.7532	0.7359
D	0.7532	0.7521	0.7366

7.4 Verdict

- **Calendar and price features improved accuracy.** WAPE fell by 0.0119 (1.57% relative), consistently across folds. This is the one external addition that clearly earns its place.
- **FEMA disaster context did not materially help.** WAPE moved by 0.0002 (0.03% relative), which is far smaller than the 0.0088 fold-to-fold spread. A difference this small cannot be distinguished from noise.

- **FRED economic context did not help.** WAPE rose by 0.0003. Monthly state unemployment is too slow-moving to explain daily deviations in a 28-day horizon.
- **Selection outcome.** The mechanical rule chose the smallest feature set within 0.3% relative of the best mean WAPE, which is arm B. The simpler model was selected on the evidence, not on preference.

Why the negative result is valuable

A common failure mode in applied data science is to add an interesting external dataset and then find a way to justify keeping it. This study measured the contribution under a fair protocol, found it immaterial, reported it in the application and in the generated results file and let the selection rule act on it. The external sources are retained where they genuinely add value - as context for interpreting shocks - and not where they do not.

8. Demand Shock Intelligence

This module is the platform's distinguishing capability. A shock is not simply high demand. It is a day on which actual demand departed from what the model could reasonably have expected, in a way that is large relative to that series' own recent forecast error and/or persistent and/or accompanied by a change in volatility or demand level.

8.1 Why the residuals come from a crisis-free model

Shock residuals are taken from arm B, which contains no FEMA or FRED features. This is deliberate. If crisis features were in the model, crisis-driven deviations would be partly absorbed into the prediction and would disappear from the residual, blinding the detector to exactly what it exists to find. Keeping crisis signals out of the model keeps them in the residual.

8.2 Score construction

Five signals are each squashed to a 0-1 range by a piecewise-linear cap and then weighted to a score out of 100. Piecewise-linear functions were chosen over smooth ones so that every point of the score is attributable to a component and the arithmetic reconciles by hand in the explanation panel.

Signal	Weight	What it measures
Standardised residual	35	Deviation relative to the series' own recent forecast error
Percentage deviation	20	Deviation relative to recent average demand
Persistence	20	Consecutive days deviating in the same direction
Volatility ratio	10	Short-term error spread against the longer-term spread
Level shift	15	Change in the demand level itself, not just the error

8.3 Guards against false positives

- A low-volume damping multiplier, so a move from zero to two units on a slow-moving item cannot register as a crisis.
- A hard zero for a zero-sale day against a near-zero forecast.
- An eligibility gate excluding series with too little non-zero history or too few residual observations, excluded series are labelled rather than scored on noise.
- A warm-up cap for the first days of the scored window, where residual history is thin.
- Residual dispersion frozen at its pre-episode value while an episode is open, so a long episode cannot inflate its own baseline and silently damp its later days.

8.4 Severity bands

Normal [0,30), Watch [30,50), Elevated [50,70), Severe [70,85), Critical [85,100].

8.5 Measured detection results

Scored over the out-of-sample window 2016-02-01 to 2016-05-22, covering 21,900 eligible series and 2,452,463 scored series-days.

Classification	Episodes	Share
Volatility Shock	23,175	43.8%
Demand Surge	15,454	29.2%
Demand Collapse	12,153	23.0%
Persistent Over-forecast	1,548	2.9%
Regime Shift	417	0.8%
Possible Regime Shift (window truncated)	127	0.2%
Persistent Under-forecast	48	0.1%

Severity band	Episodes
Critical	255
Severe	2,976
Elevated	26,286
Watch	23,405

In total 52,922 episodes were filed, with a median duration of 4 days and a maximum score of 91.7. All seven classification labels occur in the data and automated tests drive each one end-to-end to prove none is unreachable.

8.6 Real-world context, stated as coincidence

13.6% of episodes coincided with a FEMA disaster declaration active in the same state. The platform reports this as a coincidence rate and never as a causal effect.

The wording rule, enforced by test

Permitted: a FEMA disaster declaration was active within the state during this period. Not permitted: the disaster caused demand to increase. FEMA declarations are county-scoped while M5 discloses only a store's state, so the relationship is a state-level temporal coincidence. An automated test scans every generated context sentence for causal language.

8.7 Auditable explanations

Each episode carries a structured explanation payload containing the component values, the points each contributed, the guard multiplier, any warm-up cap applied, the decision trace through the classification tree and the attached real-world context. A test verifies that the arithmetic in every payload reconciles to the stored score.

9. Inventory and Business Impact

The constraint that shapes this module

M5 contains no inventory records: no on-hand stock, no lead times, no service levels and no costs. Rather than inventing values, the module separates what can be measured from what must be supplied by the user and labels the difference in the interface.

9.1 Measured from real data

These figures require no assumptions and are always displayed.

Quantity	Value
Series covered	30,490 (30,463 eligible for planning arithmetic)
Actual units sold in the holdout window	1,231,648
Forecast units over the same window	1,150,979
Revenue represented at real sell prices	\$3,900,555
Under-forecast exposure	\$1,656,859
Over-forecast exposure	\$1,389,922
Total estimated revenue exposure	\$3,046,781

Estimated Revenue Exposure dollarises forecast error using real M5 sell prices. It is deliberately not called lost revenue: M5 records units sold, so demand that was never satisfied is unobservable in the source data. Under-forecast exposure indicates potential missed sales, over-forecast exposure indicates working capital at risk of being tied up.

9.2 Requires user-supplied operating assumptions

Safety stock, reorder point, days of cover and projected stockout are computed only after the user supplies a lead time and explicitly selects a service level. Nothing is pre-populated as fact and outputs are badged as user input in the interface.

Quantity	Formula
Lead-time demand	Sum of the daily forecasts over the lead time
Safety stock	$z \times \sigma_{\text{daily}} \times \sqrt{\text{lead_time}}$
Reorder point	Lead-time demand + safety stock
Days of cover	On-hand units / mean daily forecast
Projected stockout	First day cumulative forecast exceeds on-hand units

The value of sigma is the standard deviation of that series' own out-of-sample daily forecast errors, measured across the backtest folds rather than assumed. The independence assumption behind the square-root scaling is stated explicitly and a conservative perfectly-correlated bound is displayed alongside it so the planner can see both.

9.3 Separating technical results from business assumptions

For business-case purposes the distinction matters. The measured technical results are the accuracy improvement over benchmarks, the interval coverage, the ablation finding, the shock statistics and the dollar-valued exposure at real prices. Any translation of those into inventory savings, stockout reduction or return on investment is an assumption layer that requires service levels, lead times and holding costs the data does not contain. The platform does not make that translation and neither does this report.

10. Reading a Forecast Correctly

A recurring source of misreading is the daily chart of a slow-moving product. The forecast appears as a smooth line near half a unit while actual demand jumps between zero and three. The forecast looks wrong. It is not - and understanding why is essential to using the platform, so the interface now explains it at the point of confusion.

10.1 Why a point forecast cannot track a spiky series

A product selling a few units a week contains no information about which day the sale will land. The best possible point forecast for such a series is its expected value - roughly the weekly rate divided by seven - and that line will never sit on the spikes. Demanding otherwise is asking the model to predict which customer walks in on which day.

Across the held-out window 59.6% of item-days record zero sales, so this is the normal case rather than an edge case.

10.2 The same forecast, judged at three grains

View	WAPE	What it answers
Daily, item level	0.7198	Will this exact product sell on this exact day?
Weekly, item level	0.3683	Is the weekly requirement right? (ordering grain)
Total demand per day	0.0684	Is overall demand right? (capacity, staffing)

Nothing changes between these rows except the aggregation. The identical forecast is roughly **twice as accurate** at the weekly grain a planner actually orders on and tracks total demand to within **6.8%**. The daily item-level figure is dominated by the irreducible randomness of which day a purchase falls on, not by model error.

10.3 Uncertainty is the answer, not the point forecast

For an intermittent series the honest output is a range, not a number. The platform publishes empirical P10-P90 intervals derived from out-of-sample residuals. A slow mover with an expected 0.5 units a day typically carries an interval spanning roughly zero to two units - which is precisely the statement a planner needs.

Interval property	Measured on the holdout
Design target	80%
Empirical coverage	79.4%
Actuals below P10	9.7%
Actuals above P90	10.8%
Evaluation points	853,624

A defect this analysis uncovered

Reviewing these charts revealed that the P10-P90 band had never rendered in the application. PyArrow infers a partitioned dataset's schema from its first fragment, the backtest folds carry no interval columns while the holdout does, so reading the forecast store silently dropped those columns everywhere. No error was raised - the chart simply found no interval present and drew nothing. The loader now unifies schemas across fragments and a regression test pins the behaviour. The band was the single most useful element missing from exactly the charts that looked least convincing.

10.4 Accuracy is not uniform across the assortment

Error concentrates in slow-moving products, which dominate the row count while contributing a small share of units.

Sales velocity	WAPE	Mean units/day	Share of units
Top seller	0.5110	6.32	43.8%
Fast mover	0.7070	1.89	26.3%
Steady	0.9300	0.89	18.6%
Slow mover	1.1580	0.49	8.5%
Very slow	1.3650	0.28	2.9%

A top seller is forecast more than twice as accurately as a very slow mover. Any assessment of the platform should therefore state which slice it refers to, a single headline figure averages two very different regimes.

11. Product Identification

M5 anonymises product identities. The source data contains no product names - only codes such as FOODS_3_090 - because the retailer removed them before release. Inventing names would have made the platform's central claim of using only real data false and would be the first thing an examiner questions.

Each series is therefore described by attributes measured from its own history:

Foods - Dept 3 | Value | Top seller in Foods | 130.8/day (\$1.38)

Component	Derivation
Department	The real M5 hierarchy
Price band	Median sell price, ranked within its category
Velocity	Total units sold, ranked within its category
Units per day	Measured mean daily sales
Price	Real median sell price

Ranks are taken within a category and across all stores, so the same product can be described differently in different stores. That is correct: each item-store pair is its own demand series.

A correction worth recording

Because velocity is ranked within a category, an early version labelled a hobbies product selling 0.78 units a day a Fast mover, while the same words in groceries meant 2.23 units a day. Readers reasonably interpreted the label as volume. The descriptor now names the category it is relative to and states the absolute rate, so Fast mover in Hobbies - 0.7/day cannot be confused with Top seller in Foods - 130.8/day.

11.1 Enforcement

These are display labels only. One test asserts that none of the descriptor fields appears in the model's feature list, so they cannot influence a forecast. A second scans the vocabulary for words that could not be derived from M5, so that a future edit cannot quietly introduce fabricated names.

12. Application and Service Layer

12.1 The five modules

Module	Question it answers
1. Executive Command Centre	Over the last evaluated 28 days, how did demand track forecast and where is the risk?
2. Forecasting Intelligence	How accurate is the forecast, for which products and stores and at which horizon?
3. Crisis and Demand Shock Intelligence	Where did demand break from expectation, how severe and what real events coincided?
4. Inventory and Business Impact	What does the forecast imply for stock and what is at stake in dollars?
5. Model, Explainability and Data Quality	Can this model be trusted, why does it predict what it does and is the data sound?

12.2 Interface principles

- Every displayed value is read from a pipeline artifact. No metric is hard-coded.
- Key performance indicators respond to the sidebar filters and where a figure is necessarily global it says so rather than appearing to follow the selection.
- Metrics that can mislead are annotated at the point of display, including the instability of SMAPE on intermittent demand and the meaning of item-day accuracy.
- Missing artifacts produce actionable guidance and the exact rebuild command, never a stack trace.
- Large lists are searchable and capped, with the cap stated rather than silently applied.

12.3 Service layer

A small FastAPI service exposes the same artifacts programmatically: health, metadata, forecast, shocks, metrics and an inventory analysis endpoint. Responses are typed with pydantic models, errors use a single consistent envelope, list endpoints disclose truncation and the inventory endpoint reports whether a service level was supplied by the caller or defaulted by the service.

13. Engineering Quality and Verification

13.1 Test suite

The suite comprises 119 automated tests, all passing with none skipped in the reference environment. Coverage is organised around the risks that matter most for a forecasting system.

Area	What is proven
Temporal leakage	Perturbing actuals inside the 28-day origin window leaves every feature bit-identical, with a control test proving the check is not vacuous
External signal timing	FEMA flags stay off before a declaration was issued, FRED respects the publication boundary to the day
Data provenance	The melt, the price ingest and the FEMA deduplication are checked against the raw source CSV files
Metrics	Hand-computed values, degenerate cases and pooled versus averaged aggregation
Shock scoring	Score reconciliation, determinism, guards, sigma freezing and all seven classifications driven end-to-end
Inventory	Formula correctness, lead-time clamping and that nothing is produced without user input
Service layer	Response schemas and the 200, 404, 422 and 503 paths
Graceful degradation	Every page renders guidance rather than a traceback when artifacts are absent

13.2 The central leakage proof

The most important test states the invariant directly: for a real series, actuals within the 28 days preceding and including the target date are perturbed, features are recomputed and every demand feature at the target date must be unchanged. A companion control test perturbs the day at the origin boundary and asserts the features do change, which proves the first test could actually fail. Three explicit anti-vacuity guards prevent the test from passing through an empty loop.

13.3 Independent verification

The completed project was subjected to a read-only audit that recomputed the headline metrics from the raw prediction artifacts without using project code, reconstructed shock scores from their stored components, traced processed tables back to the original CSV files and reviewed the code for leakage and the interface for misleading presentation.

Verification	Outcome
Holdout metrics recomputed independently	Matched to every printed digit
Interval coverage recomputed	Matched, no ordering violations
Shock score arithmetic reconstructed	Reconciled on every scored row
Explanation payload arithmetic	Reconciled on every episode
Melt and price ingest versus raw CSV	Exact match on all sampled cells

Verification	Outcome
Ablation fairness	Identical training rows and series in every arm
Causal language scan	No violations
Pipeline determinism	Independent re-runs reproduced results to six decimal places

Audit findings were addressed in full. These included making the executive key performance indicators respond to filters rather than mixing scopes, aligning a disclosure with what the code actually does, flagging incident end dates truncated at the data boundary, hardening the service layer contract and adding the raw-source provenance tests. The subsequent full-scale re-run reproduced every previously published figure exactly, which confirms the corrections were confined to presentation and disclosure.

14. Limitations and Responsible Use

These limitations are documented in the repository, displayed inside the application and listed by the service layer. They are stated plainly because a decision-support tool that hides its boundaries is more dangerous than one that has them.

14.1 Data limitations

- M5 contains no inventory records. Safety stock, reorder points and days of cover therefore depend on operating assumptions the user supplies.
- M5 records units sold, not demand. Unmet demand is unobservable, which is why the platform reports exposure rather than lost revenue.
- FEMA declarations are county-scoped while M5 discloses only a store's state, so disaster information is a state-level temporal coincidence.
- FRED unemployment is monthly and cannot explain a daily deviation, it functions as slow-moving background context.
- Weather data is out of scope in this version.

14.2 Methodological limitations

- A seven-day-ahead forecast sees demand only as of 28 days earlier, the deliberate cost of the single-model direct design.
- FEMA active flags switch off using recorded incident end dates, which are knowable only retrospectively, the onset is strictly point-in-time. This affects only the ablation arms that contain FEMA features, which showed no material gain in any case.
- The model and the naive benchmarks are scored on slightly different rows, because the model requires a feature warm-up period that the benchmarks do not.
- Shock detection is retrospective: residuals exist only where the model forecast out-of-sample, so every shock shown is a measured historical deviation rather than a prediction of future risk.
- Hierarchical aggregation is bottom-up, no formal forecast reconciliation is performed.
- No causal inference is attempted. External signals are treated as association only.
- The demand shock score is a metric defined by this project, not a validated industry standard and the application says so on the page.

14.3 Responsible use

The platform is decision support. Forecast uncertainty is displayed rather than hidden, poorly performing segments are surfaced rather than buried and the ablation verdict is generated from the numbers so it cannot be quietly softened. Human review belongs between any output here and an ordering decision.

15. Reproduction

15.1 Environment

Python 3.11 or later. Install dependencies, then run the dependency smoke test, which exercises every library call the pipeline relies on so environment problems surface before anything else runs.

```
pip install -r requirements.txt
python scripts/smoke_deps.py
```

15.2 Build and serve

```
python scripts/run_pipeline.py --mode development
streamlit run app/Home.py
uvicorn api.main:app --reload
python -m pytest
```

Development mode uses a real subset of the M5 data - one store, one category, a fixed number of real items - so the pipeline completes in minutes. It is a subset of real data, never generated data. Full mode processes all 30,490 series.

15.3 Measured run characteristics

Stage	Full-scale duration
Data preparation and feature engineering	Approximately 5 minutes
Training, backtesting and ablation	Approximately 55 minutes
Shock scoring and inventory analysis	Approximately 11 minutes
Complete test suite	Under 2 minutes

The deploy model was fitted on 21,561,935 training rows with a 730-day training window, converging at iteration 254.

16. Conclusion

DemandShock demonstrates an end-to-end applied data science workflow: data engineering at scale, time-series forecasting under strict temporal discipline, anomaly and regime detection, external data integration evaluated on its merits, explainability, inventory decision support, software engineering practice and executive communication.

The forecasting model measurably outperforms the benchmark that shares its information set, its prediction intervals are well calibrated at scale and its shock detection produces auditable, individually explainable episodes. Equally important, the project reports honestly where its hypothesis failed: external crisis and economic context did not materially improve point forecasts, the selection rule acted on that evidence and the finding is displayed rather than concealed.

The result is a platform whose numbers can be traced from the interface back through the artifacts to the original source files and which states its limitations as clearly as its results.